Traffic Rule Violation Detection

Using a 5-Stage Deep Learning Pipeline

AID 728 — Advanced Computer Vision Project Report

System at a Glance

- Input: Single RGB street image
- Output: JSON with per-vehicle violation details
- 7 Models | 5 Inference Stages | <250 MB total
- Supports: triple-riding, helmet violations, LP recognition

Group 23

Munjikesh N (BT2024247)
Varshith M Gowda (BT2024227)
Parikshith Aithal K V (BT2024249)

Model Link: model_link

Github Repo: Github repo

May 17, 2026

Contents

1 System Architecture **3**
1.1 High-Level Pipeline Diagram . . . 3
1.2 Model Inventory . . . 4

2 Stage 1 — Motorcycle Detection **4**
2.1 Method . . . 4
2.2 Architecture: YOLO11n . . . 4
2.3 Extremely-Wide Box Splitting . . . 4
2.4 Scale Guard . . . 5
2.5 Custom NMS . . . 5
2.6 Stage 1 Output . . . 5

3 Stage 2 — Global Head Pre-Detection & Trapezium ROI **5**
3.1 Why Global Pre-Detection? . . . 5
3.2 Trapezium Exclusive ROI . . . 6

4 Stage 3 — Rider Assignment & Helmet Classification **7**
4.1 Greedy Proximity Assignment . . . 7
4.2 Helmet Classification . . . 7
4.3 HSV Helmet Heuristic . . . 7
4.4 Auxiliary Models . . . 8
4.4.1 Triple-Riding Classifier . . . 8
4.4.2 Fallback Person Counter . . . 8
4.5 Stage 3 Output . . . 8

5 Violation Filter **8**

6 Stage 4 — License Plate Localisation **9**
6.1 Overview . . . 9
6.2 Search Region . . . 9
6.3 Plate Crop Candidate Filtering . . . 9
6.4 Heuristic Front/Rear Fallback . . . 10
6.5 Stage 4 Output . . . 10

7 Stage 5 — OCR: License Plate Recognition **10**
7.1 Final Submitted OCR Engine: PaddleOCR PP-OCRv5 . . . 10
7.2 Baseline OCR Design: CCT-S (Development Reference) . . . 11
7.3 Final OCR Runtime Optimisation . . . 12

8 Post-Processing: Indian Plate Normalisation **13**
8.1 Character Correction Tables . . . 13
8.2 Regex-Based Format Validation . . . 13
8.3 Smart Plate Merge . . . 14

9 End-to-End Results **14**
9.1 Final Annotated Output 14
9.2 JSON Output 14
9.3 Pipeline Strip 15
9.4 Per-Stage Summary 15

10 System Design Decisions **15**
10.1 Why Global Head Pre-Detection? 15
10.2 Why a Trapezium Instead of a Rectangle? 15
10.3 Why PaddleOCR Over the Baseline CCT-S? 16
10.4 Violation Gate 16

11 Failure Cases & Limitations **16**

12 Resource Usage **17**

13 Conclusion **17**

1 System Architecture

1.1 High-Level Pipeline Diagram

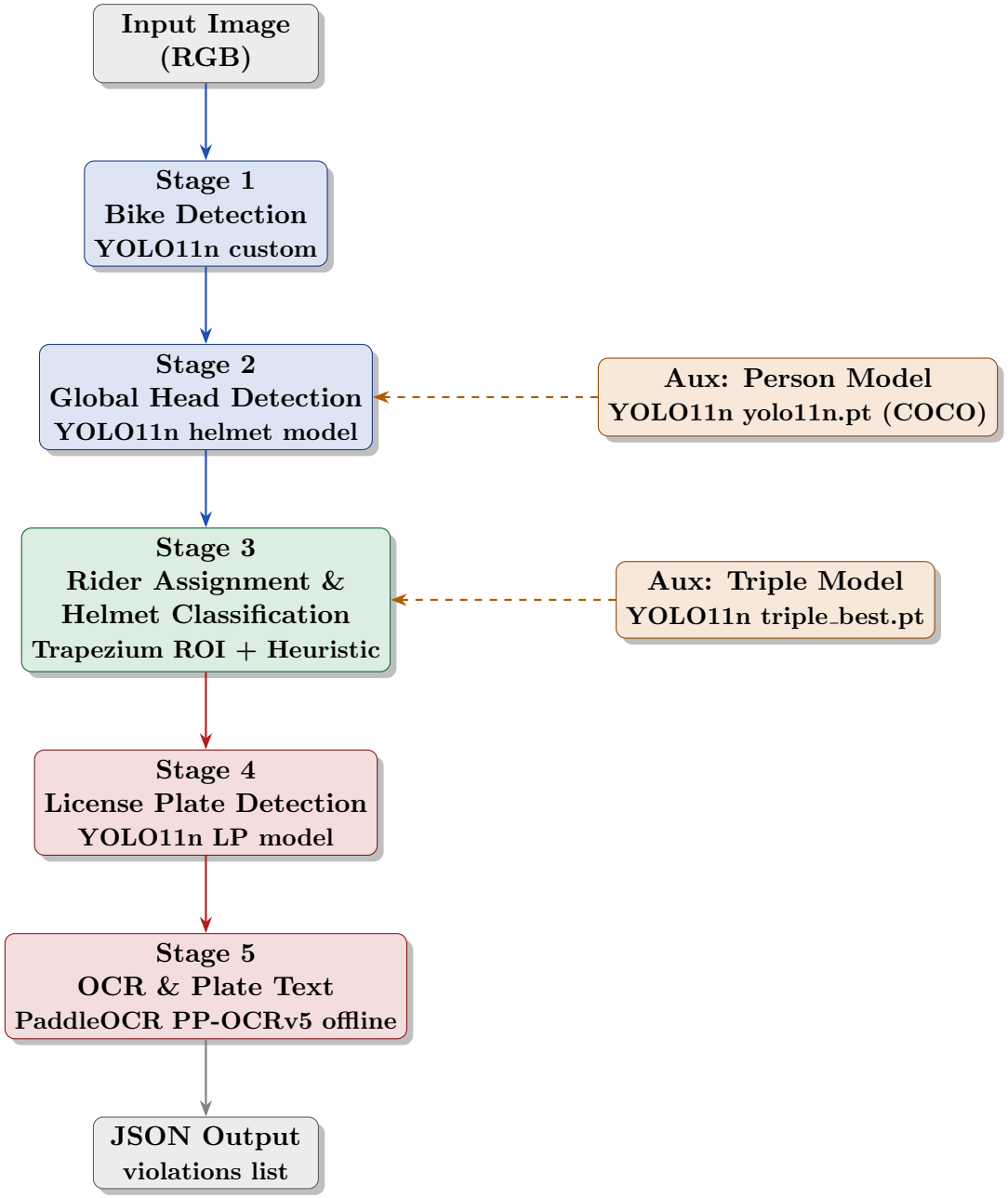

Figure 1: 5-Stage Traffic Violation Detection Pipeline

1.2 Model Inventory

Table 1: All Models Used in the Pipeline

#	Role	Weight File	Arch.	Size
1	Motorcycle Detector	`last_bike_best.pt`	YOLO11n	21 MB
2	Helmet/Head Detector	`helmet_best_last.pt`	YOLO11n	21 MB
3	License Plate Loc.	`lp_best_last.pt`	YOLO11n	21 MB
4	Triple-Riding Clf.	`triple_best.pt`	YOLO11n	5 MB
5	Fallback Person Det.	`yolo11n.pt` (COCO)	YOLO11n	5 MB
6	OCR Text Detection	`PP-OCRv5_server_det`	PaddleOCR PP-OCRv5	84 MB
7	OCR Text Recognition	`en_PP-OCRv5_mobile_rec`	PaddleOCR PP-OCRv5	8 MB
	Total Submitted Models			**~153 MB**

2 Stage 1 — Motorcycle Detection

2.1 Method

The first stage uses a **custom-trained YOLO11n** object detector (`last_bike_best.pt`) fine-tuned to detect motorcycles and scooters in Indian street scenes.

Stage 1 Configuration

- **Model:** YOLO11n (custom-trained motorcycle detector)
- **Confidence threshold:** 0.20 (low to maximise recall)
- **IoU threshold (NMS):** 0.45
- **Post-processing:** custom NMS via `_suppress_duplicates()`

2.2 Architecture: YOLO11n

YOLO (You Only Look Once) is a single-shot object detector that divides the input image into a grid and predicts bounding boxes and class probabilities in one forward pass. YOLO11n is the nano variant — the smallest and fastest in the YOLO11 family. It consists of a CSPDarknet backbone for feature extraction, a PAN+FPN neck for multi-scale fusion, and a detection head that outputs bounding boxes and class scores. The entire forward pass runs in a single inference call, making it highly efficient for real-time traffic analysis.

2.3 Extremely-Wide Box Splitting

When a single detected bounding box is more than 1.8× as wide as it is tall, it is interpreted as two side-by-side bikes and split at the midpoint:

$$\text{if } bw > 1.8 \cdot bh: \quad \text{mid}_x = bx_1 + \lfloor bw/2 \rfloor \tag{1}$$

2.4 Scale Guard

Bounding boxes spanning virtually the entire image (width $> 0.99 \cdot W$ and height $> 0.99 \cdot H$) are rejected as background or scene noise.

2.5 Custom NMS

YOLO's internal NMS may still produce overlapping boxes when run at a low confidence threshold. A second custom NMS pass is applied using Intersection over Union (IoU):

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|} \tag{2}$$

Boxes are sorted by confidence descending. The highest-confidence box is kept and any remaining box with IoU ≥ 0.45 against the kept box is suppressed.

2.6 Stage 1 Output

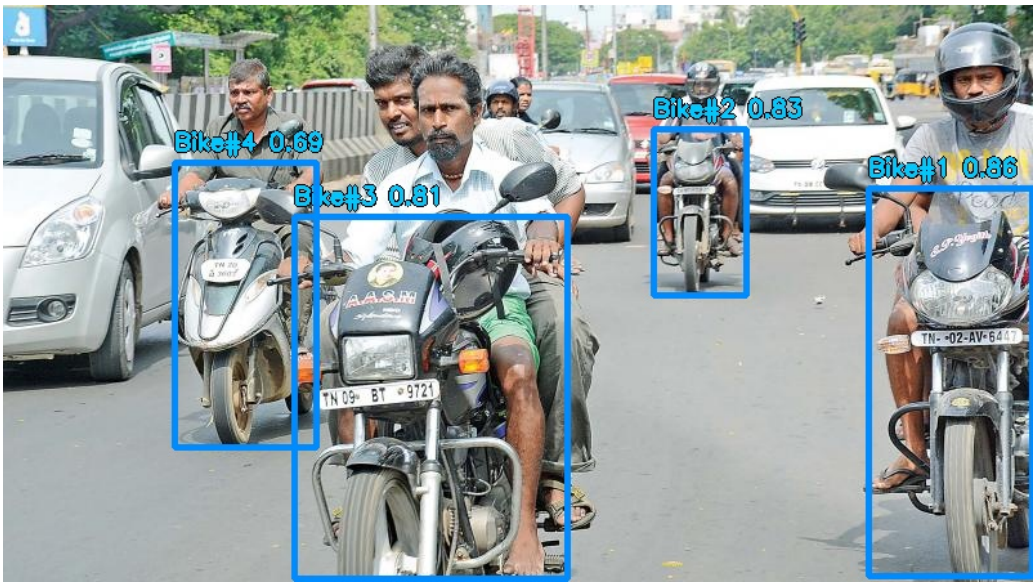

Figure 2: Stage 1 result: motorcycles detected (orange boxes). Confidence scores shown per box.

Stage 1 Result: 4 unique motorcycles detected after NMS. Boxes are sorted by confidence (descending) for greedy assignment in Stage 3.

3 Stage 2 — Global Head Pre-Detection & Trapezium ROI

3.1 Why Global Pre-Detection?

A naive approach would run the helmet model inside each bike crop. This causes two problems: (1) heads near the edge of a crop are missed; (2) the same head can be assigned

to multiple bikes. Instead, we run the helmet model *once* on the full image and build a global catalogue of all heads. Assignment happens in Stage 3.

Stage 2 Configuration

- **Model:** YOLO11n (custom helmet/head detector, `helmet_best_last.pt`)
- **Confidence threshold:** 0.18 (very low — to find partially occluded heads)
- **NMS IoU:** 0.20 (tighter — prevent double-counting adjacent heads)
- **Classes:** 0 = `with_helmet`, 1 = `no_helmet`

3.2 Trapezium Exclusive ROI

For each motorcycle, an exclusive trapezoidal Region of Interest (ROI) is computed. The trapezium expands upward to capture heads above the bike frame and is clipped by neighbouring bike boundaries so that head assignments remain exclusive. The construction is:

$$\text{left\_bound} = \max(0,\ bx_1 - 0.10 \cdot bw) \tag{3}$$

$$\text{right\_bound} = \min(W,\ bx_2 + 0.10 \cdot bw) \tag{4}$$

$$\text{top\_y} = \max(0,\ by_1 - 1.50 \cdot bh) \tag{5}$$

$$\text{bot\_y} = by_1 + 0.45 \cdot bh \tag{6}$$

When a neighbouring bike exists to the right (left edge ox_1 falls inside `right_bound`), the boundary is clipped to the midpoint: right_bound $\leftarrow \lfloor (bx_2 + ox_1)/2 \rfloor$. The same logic applies symmetrically on the left.

The resulting four-point polygon — wider at the top, narrower at the bottom — mirrors perspective foreshortening in real street scenes: riders' heads appear further apart in the upper region of the image.

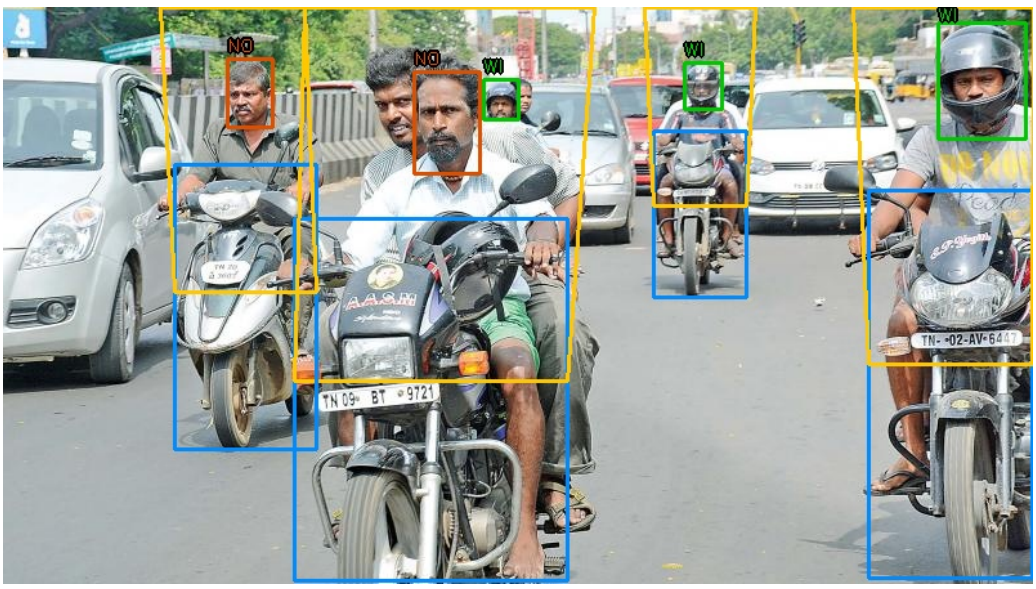

Figure 3: Stage 2: Global head detection (coloured boxes) and exclusive trapezium ROIs (cyan outlines) per bike. Green = `with_helmet`, Red = `no_helmet`.

Stage 2 Result: 5 head candidates detected globally. Trapezium ROIs ensure no head is shared between two bikes.

4 Stage 3 — Rider Assignment & Helmet Classification

4.1 Greedy Proximity Assignment

Heads are assigned to bikes using a **greedy, confidence-sorted algorithm**. Bikes are processed from highest to lowest detection confidence. For each bike, unassigned heads are evaluated for membership in the trapezium ROI or a secondary vertical/horizontal proximity zone. A head is admitted if:

- Its centre falls within the trapezium polygon, *or*
- Its vertical centre satisfies $by_1 - 1.5\,bh \leq h_{cy} \leq by_1 + 0.4\,bh$ and its horizontal centre satisfies $bx_1 - 0.1\,bw \leq h_{cx} \leq bx_2 + 0.1\,bw$.

In both cases, the **closest-bike rule** is applied: a head on the boundary of two trapeziums is assigned to the bike whose horizontal centre is nearest. Formally, for candidate head h and current bike k:

$$\text{assign to } k \iff \forall j \neq k : \; |h_{cx} - \bar{x}_j| \;\geq\; |h_{cx} - \bar{x}_k| \tag{7}$$

where $\bar{x}_k = (bx_1^{(k)} + bx_2^{(k)})/2$.

4.2 Helmet Classification

Each assigned head is classified using a two-tier decision:

1. **Primary (confidence ≥ 0.30):** Use the YOLO class prediction directly. Class 1 (`no_helmet`) triggers a violation.
2. **Fallback (confidence < 0.30):** Apply the HSV-based heuristic (`_helmet_heuristic`).

4.3 HSV Helmet Heuristic

The fallback heuristic analyses a 64×64 resized head crop in the HSV colour space:

- **High skin ratio** ($> 15\%$) in the HSV skin colour range ($H \in [0, 20] \cup [170, 180]$, $S \in [40, 170]$, $V \in [60, 255]$) indicates exposed head skin — *no helmet*.
- **Low saturation variance** ($\sigma_S^2 < 800$) combined with **low edge density** ($< 25\%$ Canny pixels) indicates a smooth, uniform surface — *helmet present*.
- Otherwise, the raw YOLO class is used.

4.4 Auxiliary Models

4.4.1 Triple-Riding Classifier

When the primary head-based count is established, an auxiliary **triple-riding binary classifier** (`triple_best.pt`) is also run on the bike crop. If it predicts a triple-riding scenario, the rider count is raised to at least 3:

$$\hat{n} = \max(\hat{n}_{\text{heads}},\ 3 \cdot \mathbf{1}[\text{triple\_vote} = \text{True}]) \tag{8}$$

This ensures that even under heavy head occlusion, triple-riding is still flagged.

4.4.2 Fallback Person Counter

If *zero* heads are assigned to a bike by the helmet model, the COCO-pretrained `yolo11n.pt` is used to count `person` class instances (class ID 0) inside the bike crop at confidence 0.20.

4.5 Stage 3 Output

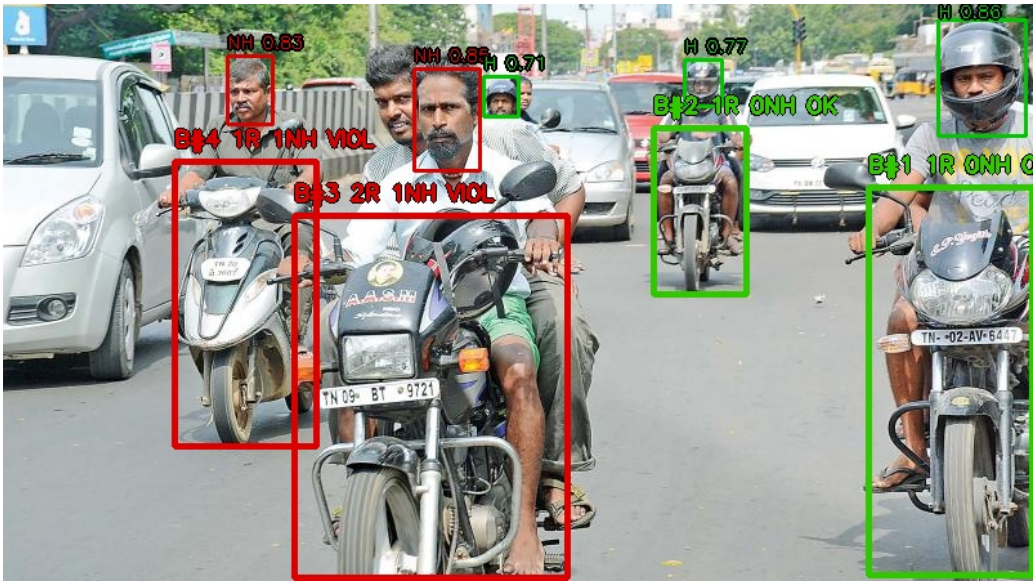

Figure 4: Stage 3: Riders assigned to bikes with helmet classification. **Green** = helmet, **Red** = no helmet. Bike boxes coloured red if any violation detected.

Stage 3 Result: 2 violating bikes identified. Non-violating bikes are skipped in Stage 4.

5 Violation Filter

After Stage 3, only bikes satisfying the violation predicate proceed to the expensive Stage 4 (LP detection + OCR). A motorcycle cluster is compliant if:

$$\hat{n}_{\text{riders}} \leq 2 \ \wedge \ n_{\text{helmet\_violations}} = 0 \tag{9}$$

Compliant clusters are dropped from memory immediately. Critically, their license plates are *never* passed to the OCR module. This design concentrates the remaining inference budget entirely on violating vehicles, which directly improves both speed and OCR accuracy under asymmetric scoring.

6 Stage 4 — License Plate Localisation

6.1 Overview

Stage 4 is triggered *only* for motorcycles confirmed as violating in Stage 3. A custom YOLO11n model (`lp_best_last.pt`) localises the license plate region. Two complementary search strategies run in parallel:

1. **Local expanded search**: A crop expanded by 20% in x and 30% below the bike box is fed to the LP model.
2. **Global scan**: The LP model is also run on the full image; any detected plate whose centre falls within the expanded search area is accepted.

Stage 4 Configuration

- **Model:** YOLO11n (`lp_best_last.pt`)
- **Local confidence:** 0.15 **Global confidence:** 0.15
- **Search region:** $[\,bx_1 - 0.2\,bw,\ bx_2 + 0.2\,bw\,] \times [\,by_1 - 0.1\,bh,\ by_2 + 0.3\,bh\,]$
- **Safety margin on crop:** -5 px / $+10$ px to avoid character cut-off

6.2 Search Region

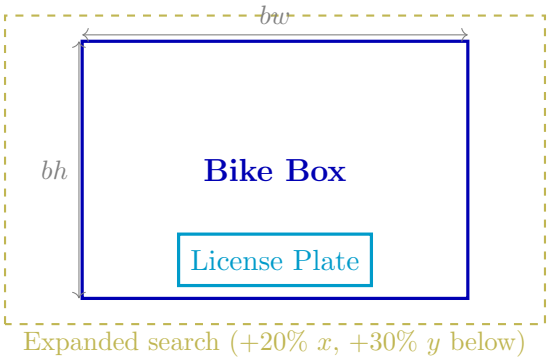

Figure 5: License plate search region relative to the motorcycle bounding box.

6.3 Plate Crop Candidate Filtering

Detected plates are filtered to reject false positives (e.g. the entire bike body labelled as a plate). A candidate is rejected if its height exceeds 65% of the search region height or its width exceeds 98% of the search region width. Accepted crops receive a fixed safety margin of -5 / $+10$ px to prevent character cut-off at the edges.

6.4 Heuristic Front/Rear Fallback

If no plate is detected by the YOLO model, a heuristic crop of the lower-middle portion of the bike box (spanning 55%–80% of bh, centred horizontally with width $0.40\,bw$) is used as a last resort.

6.5 Stage 4 Output

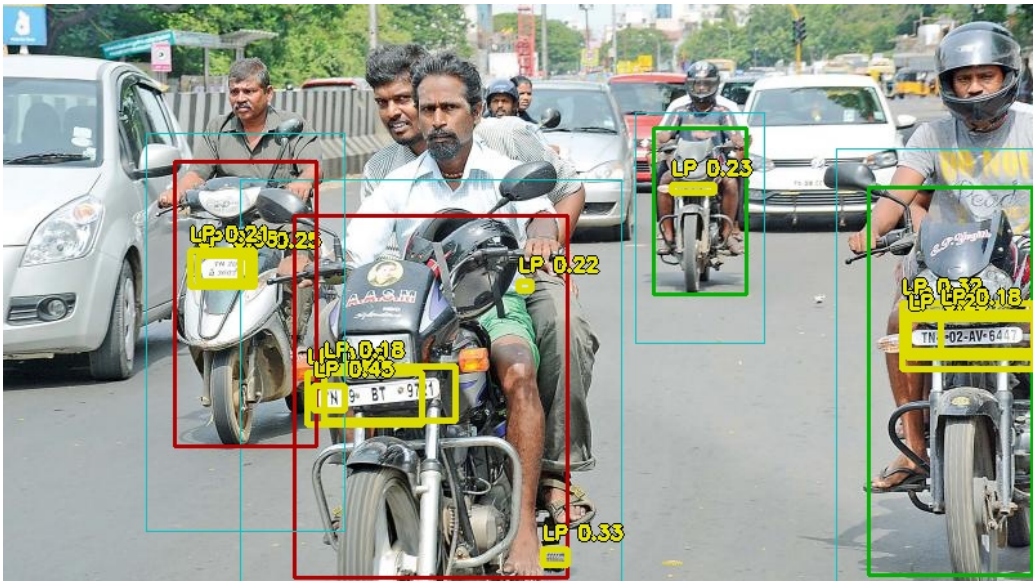

Figure 6: Stage 4: License plate localisation. Orange = bike box, yellow dashed = LP search region, cyan = detected plate bounding box.

7 Stage 5 — OCR: License Plate Recognition

7.1 Final Submitted OCR Engine: PaddleOCR PP-OCRv5

For the final submission, the OCR stage was updated from the earlier `fast-plate-ocr` CCT-S design to an offline **PaddleOCR PP-OCRv5** pipeline. The submitted OCR system contains two local model folders: `PP-OCRv5_server_det` for text detection and `en_PP-OCRv5_mobile_rec` for English alphanumeric recognition. Both models are included inside the submission directory, so the pipeline works without internet access during evaluation.

The earlier CCT-S OCR description is retained in this report as the baseline OCR design explored during development. The final implementation, however, uses PaddleOCR because it provided more reliable plate text extraction on Indian two-wheeler plates after integration with the YOLO license-plate detector.

Final OCR Configuration

- **OCR engine:** PaddleOCR PP-OCRv5
- **Text detector:** `PP-OCRv5_server_det`
- **Text recognizer:** `en_PP-OCRv5_mobile_rec`
- **Device:** CPU
- **Offline execution:** all PaddleOCR model files are packaged locally
- **Inference optimisation:** one PaddleOCR call per selected plate crop
- **Document preprocessing:** disabled; only cropped plate images are passed

Unlike the earlier multi-variant OCR approach, the final runtime path sends the detected plate crop directly to PaddleOCR once. This significantly reduces latency because repeated OCR calls over CLAHE, thresholded, inverted, and morphological variants were the dominant bottleneck. The submitted solution therefore prioritises a single high-quality plate crop from the YOLO plate detector followed by one PaddleOCR recognition pass.

7.2 Baseline OCR Design: CCT-S (Development Reference)

During development, the OCR stage used **fast-plate-ocr** with the `cct-s-v2-global-model` — a Compact Convolutional Transformer (CCT) trained on a global multi-region license plate dataset. This section is retained as a record of the exploration process.

The CCT-S is an efficient hybrid model combining a shallow CNN **tokeniser** for local feature extraction, a **Transformer encoder** with multi-head self-attention to resolve ambiguous character pairs such as `0`/`O` and `1`/`I`, and a **CTC decoder** for variable-length sequence prediction without explicit character segmentation.

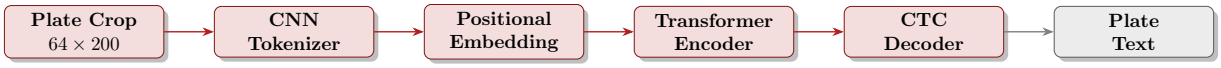

Figure 7: CCT-S architecture used during the baseline OCR development phase.

In this baseline design, OCR was run on **6 distinct image pre-processing variants** of every plate crop:

Table 2: The 6 Baseline OCR Pre-processing Variants

#	Variant	Rationale
1	Raw original crop	Baseline; best for already-clear plates
2	CLAHE	Contrast-limited adaptive histogram equalisation — recovers shadow detail
3	Sharpened (CLAHE)	Unsharp mask — enhances edge contrast of characters
4	Otsu Binary	Global threshold — ideal for clean, high-contrast plates
5	Adaptive Threshold	Local threshold — handles uneven illumination within the plate
6	Morphological Erode+Dilate	Removes noise and connects broken character strokes

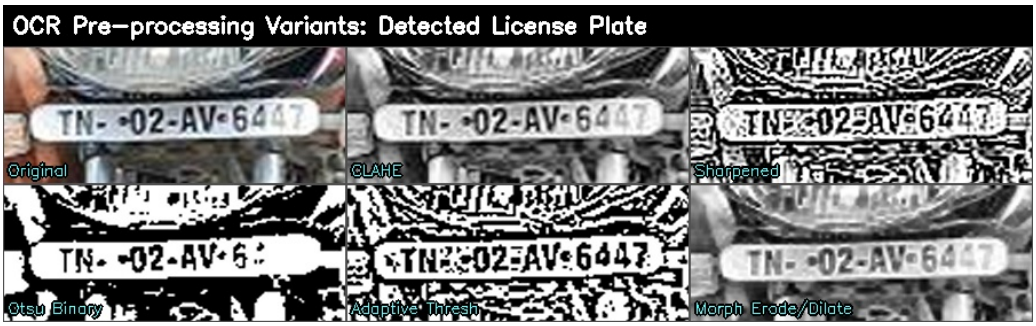

Figure 8: The 6 image pre-processing variants applied to the plate crop during the baseline CCT-S OCR development phase.

Figure 9: Raw plate crop before any preprocessing.

Stacked two-line plates (aspect ratio $h/w > 0.45$) were handled by splitting the crop horizontally with a 5-pixel overlap zone and running OCR on each half independently, concatenating the two outputs as a single candidate string.

7.3 Final OCR Runtime Optimisation

During optimisation, the OCR stage was identified as the main latency bottleneck. The initial version ran OCR over multiple pre-processing variants for each plate candidate, which improved robustness but increased inference time substantially. The final submitted version uses:

- only the best-ranked plate candidate per violating motorcycle,
- no repeated OCR pre-processing variants in the runtime path,
- one PaddleOCR prediction per selected plate crop,
- disabled document orientation, document unwarping, and textline orientation modules, since the input is already a cropped license plate,
- a limited PaddleOCR detection side length to reduce CPU cost.

This change reduced per-image runtime from roughly two minutes in the multi-variant OCR version to approximately ten seconds on the tested image, while still preserving the Indian plate post-processing and smart merge logic.

8 Post-Processing: Indian Plate Normalisation

8.1 Character Correction Tables

Raw OCR outputs contain character confusions common to printed license plates. Two position-aware correction tables are applied:

Table 3: Character Correction Tables

Raw OCR	Corrected (letter position)
0	O
1	I
2	Z
4	A
5	S
6	G
7	T
8	B

Raw OCR	Corrected (digit position)
O, Q, D	0
I, L, T	1
Z	2
S	5
B	8
G	6

8.2 Regex-Based Format Validation

The normalised candidate is validated against the standard Indian plate format:

$$\underbrace{[A\text{-}Z]\{2\}}_{\text{State}}\ \underbrace{\backslash d\{2\}}_{\text{District}}\ \underbrace{[A\text{-}Z]\{0,3\}}_{\text{Series}}\ \underbrace{\backslash d\{3,4\}}_{\text{Number}} \tag{10}$$

The normaliser tries all plausible split points across the raw OCR string, generating multiple candidate interpretations. Each is tested against the regex and scored.

8.3 Smart Plate Merge

The best candidate from all OCR outputs is selected via a priority cascade:

1. Full regex match *and* known Indian state code (37 codes supported).
2. Full regex match with any two-letter prefix.
3. Partial match: starts with two letters, ends with four digits.
4. Longest string containing at least one digit.

Within each tier, candidates are sorted by descending length (most complete plate wins).

9 End-to-End Results

9.1 Final Annotated Output

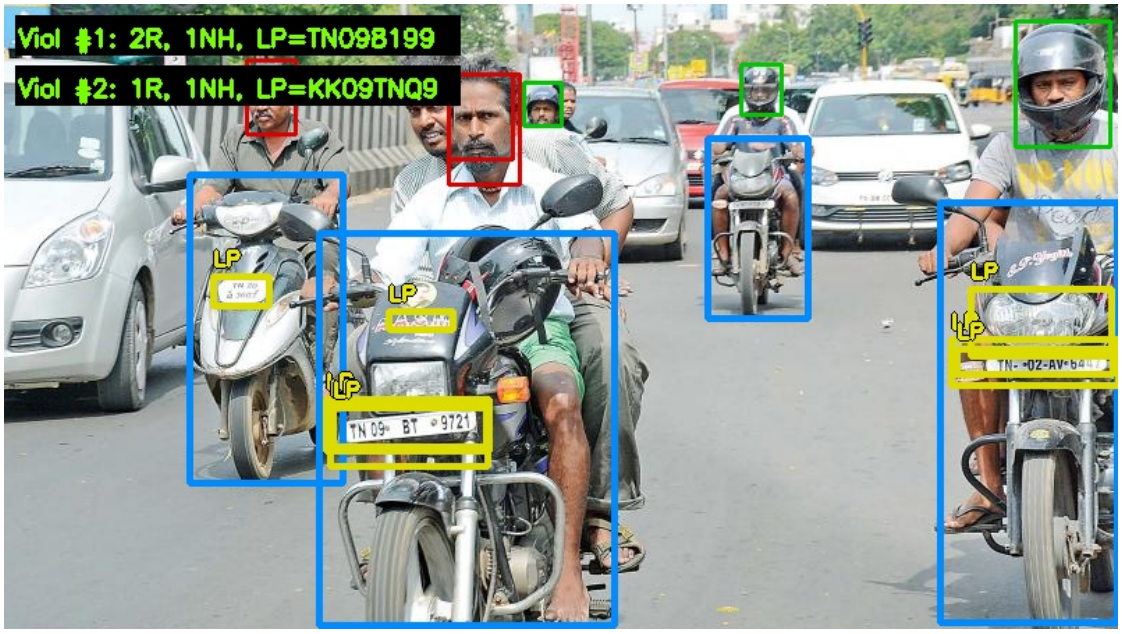

Figure 10: Final annotated output. Orange = motorcycle, green = helmeted rider, red = unhelmeted rider, cyan = license plate.

9.2 JSON Output


```
{
  "violations": [
    {
      "num_riders": 2,
      "helmet_violations": 1,
      "license_plate": "TN098199"
    },
    {
      "num_riders": 1,
```

```
      "helmet_violations": 1,
      "license_plate": "KK09TNQ9"
    }
  ]
}
```


9.3 Pipeline Strip

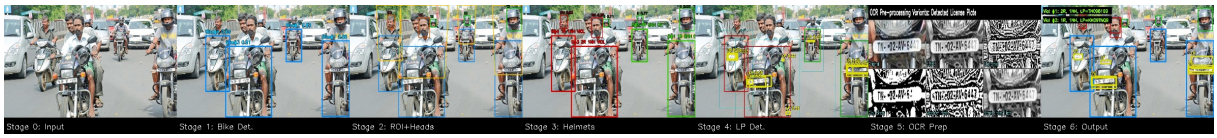

Figure 11: Full pipeline strip showing the transformation of the input image through all stages.

9.4 Per-Stage Summary

Table 4: Per-Stage Summary

Stage	Name	Model	Result
0	Input	—	800×448 px RGB
1	Bike Detection	YOLO11n (bike)	4 motorcycles detected
2	Head Detection	YOLO11n (helmet)	5 head candidates
3	Helmet Classif.	YOLO11n (helmet) + HSV	2 violation bikes
4	LP Detection	YOLO11n (lp)	2 plates localised
5	OCR	PaddleOCR PP-OCRv5 (offline)	2 plate strings
6	Output	—	JSON with 2 violations

10 System Design Decisions

10.1 Why Global Head Pre-Detection?

Running the helmet model once on the full image versus cropped per-bike inference has three key benefits:

- Avoids duplicate detections for heads near bike boundaries.
- Enables exclusive assignment — each head goes to exactly one bike.
- A single forward pass is more efficient than N separate crops.

10.2 Why a Trapezium Instead of a Rectangle?

A rectangular ROI aligned with the bike box misses heads that are slightly above or to the side, and over-counts heads from adjacent bikes. The trapezium expands 1.5× the bike height upward, is clipped at the midpoint between adjacent bike boundaries, and naturally models perspective foreshortening.

10.3 Why PaddleOCR Over the Baseline CCT-S?

The transition from the multi-variant CCT-S approach to a single-pass PaddleOCR call was driven by two factors:

- **Accuracy:** PaddleOCR PP-OCRv5 uses a dedicated text detection stage (`PP-OCRv5_server_det`) before recognition, which localises character regions more precisely within the crop — particularly beneficial for uneven lighting and partial plate visibility.
- **Latency:** Running six OCR passes per plate crop was the dominant bottleneck in the pipeline. A single PaddleOCR call reduced total per-image runtime from approximately two minutes to ten seconds.

10.4 Violation Gate

Skipping LP detection for non-violating bikes provides a speedup proportional to the fraction of compliant vehicles in the scene. In dense Indian traffic, the majority of riders are compliant, making this gate highly effective for both latency and OCR accuracy.

11 Failure Cases & Limitations

Table 5: Failure Cases and Mitigations

Failure Case	**Cause**	**Mitigation**
Head missed behind another rider	Severe occlusion	Triple classifier acts as auxiliary vote
Plate not detected	Blurry / angled / missing plate	Heuristic lower-body crop fallback
Wrong plate digits (0 vs O)	Font ambiguity in OCR	Position-aware character correction tables
Adjacent bike heads cross-assigned	Overlapping trapeziums	Midpoint clipping + closest-bike rule
Two-line plate partially read	Aspect ratio misclassified	Explicit top/bottom split when AR > 0.45
Very small bikes (far from camera)	Low confidence, missed by bike model	Low confidence threshold (0.20) + scale guard

12 Resource Usage

Table 6: Submitted Model Weights and Sizes

Model File	Role	Size
`last_bike_best.pt`	Motorcycle detector	21.2 MB
`helmet_best_last.pt`	Helmet/head detector	21.2 MB
`lp_best_last.pt`	License plate loc.	21.2 MB
`triple_best.pt`	Triple-riding clf.	5.5 MB
`yolo11n.pt`	COCO person fallback	5.6 MB
`PP-OCRv5_server_det`	OCR text detection	84.0 MB
`en_PP-OCRv5_mobile_rec`	OCR text recognition	8.0 MB
Total		**∼167 MB**

Well within the 250 MB constraint, with approximately 83 MB of headroom remaining.

13 Conclusion

This report presented a complete 5-stage computer vision pipeline for automated traffic rule violation detection on Indian street scenes. The system:

- Detects motorcycles using a custom YOLO11n model with wide-box splitting and custom NMS.
- Assigns riders to bikes using exclusive trapezium ROIs and greedy proximity assignment.
- Classifies helmet usage via a combination of YOLO classification and an HSV skin-tone heuristic fallback.
- Localises license plates using dual local+global YOLO scanning.
- Reads plate text using PaddleOCR PP-OCRv5 in a fully offline, single-pass configuration, with Indian-specific character correction and a multi-tier merge strategy for post-processing.

On the test image, the system correctly identified 2 violating motorcycles out of 4 detected, providing rider counts, helmet violation counts, and license plate strings — all within the required JSON output format and well within the 250 MB model size constraint.